DAY 03 | PRE-READ

Blueprints and Good Habits

A light read before Day 3. About 15 minutes, no code required.

Prepared by **Ahmed** | linkedin.com/in/ahmed-al-imaad

Why read this?

As programs grow, they need structure, the same way a growing company needs teams and roles. Tomorrow you will learn object-oriented programming (OOP), a way to organise code around real things, plus habits that keep code tidy.

1. Blueprints and buildings

An architect draws one blueprint for a house. A builder can use it to build many houses. Each house has the same rooms, but different families, paint colours and furniture. In programming, the blueprint is called a **class**, and each house is an **object**.

Real world	Things it has (data)	Things it does (actions)
Bank account	owner, balance	deposit, withdraw
Car	brand, colour, speed	start, brake, accelerate
Online order	items, total, status	add item, pay, cancel

2. Families of things (inheritance)

A savings account and a current account are both bank accounts. They share most features but differ in a few rules. Instead of describing everything twice, we describe the shared parts once and only the differences for each type. That is inheritance.

3. Planning for things going wrong

A good restaurant has a plan for when an ingredient runs out: offer something else instead of closing the kitchen. Programs need the same. **Exception handling** lets a program notice a problem, deal with it politely, and keep running.

4. Good habits: one job each

In a well run kitchen, the chef cooks, the cashier takes payments and the cleaner cleans. If one person did everything, any change would cause chaos. Good code follows the same idea, summed up in five principles called **SOLID**. The most important one: every part should have one clear job.

5. Helpful tools you will meet

- **Modules**: code split into separate files, like chapters in a book.

- **NumPy**: a tool for doing maths on thousands of numbers at once, very quickly.
- **Decorators**: gift wrapping for functions. The gift stays the same, but something is added around it.
- **Generators**: a tap instead of a bucket. You get water when you need it, not all at once.

6. Think about these before class

- Pick an app you use daily. What "things" does it manage, and what can each thing do?
- Think of a time a website handled an error well, and one where it handled it badly. What was the difference?
- Why might giving one person too many jobs cause problems at work?

Monozukuri (ものづくり): the craft of making things well